Real-Time Market Microstructure Analyzer

Project Report

Author:	Aarya Parekh
Institution:	Indian Institute of Technology Bombay
Programme:	B.Tech, Semester 5
Date:	16 August 2026
Repository:	github.com/aariiparekh3012-collab/quantproject2

1. Overview

This document describes the design and implementation of the Real-Time Market Microstructure Analyzer — a full-stack web application that ingests live Indian equity market data (NSE/BSE) via WebSocket, computes microstructure analytics in real time, and presents them through an interactive dashboard.

The system implements five analytics modules: bid-ask spread decomposition, Order Flow Imbalance following Cont, Kukanov & Stoikov (2014), session VWAP with deviation bands, volume profiling with tick-rule classification, and market impact estimation (Kyle's Lambda, Amihud Illiquidity). A statistical anomaly detector identifies spread blow-ups, volume spikes, and order book imbalance extremes using rolling z-score methods.

Item	Detail
Source files	43
Backend modules	18 Python modules
Frontend components	8 React components
Analytics computed per tick	14 metrics
Data source	Angel One SmartAPI / Mock generator

2. Technology Stack

Layer	Technology	Rationale
Data ingestion	Python, WebSocket	Native async, broker API compatibility
Analytics engine	NumPy, Pandas, SciPy	Vectorized numerical computation
Backend API	FastAPI	Async support, native WebSocket, auto-generated docs
Cache	Redis (in-memory fallback)	Sub-millisecond reads for live state
Storage	Apache Parquet	Columnar format, partitioned by date/symbol
Frontend	React 18, Recharts, Vite	Live charting, fast development builds
Deployment	Render, Vercel	Free-tier CI/CD

3. System Architecture

The system follows a three-tier architecture separating data ingestion, analytics computation, and presentation.

The **ingestion layer** connects to the broker WebSocket API (or a mock data generator) and normalises incoming 5-level order book snapshots into a standard format. The **analytics layer** processes each snapshot through spread, OFI, VWAP, volume, and anomaly detection modules, all operating in streaming (online) mode with no batch dependencies. The **presentation layer** consists of a FastAPI server that exposes REST endpoints for historical queries and WebSocket endpoints for real-time streaming to a React frontend.

Data flows unidirectionally: broker API to ingestion service to analytics engine to API server to dashboard clients. Raw ticks are persisted in Parquet files partitioned by symbol, date, and 15-minute time bucket. The latest order book state is cached in Redis (with automatic in-memory fallback) for sub-millisecond access.

4. Analytics Modules

4.1 Bid-Ask Spread Analysis

Computes three spread variants: quoted spread ($ask_1 - bid_1$), relative spread normalised by midprice, and depth-weighted spread across all five visible book levels. Spread time series feed into the anomaly detector.

4.2 Order Flow Imbalance

Implements the Cont-Kukanov-Stoikov (2014) event-contribution framework. Tracks bid/ask price and quantity transitions between consecutive snapshots. Rolling OFI is maintained across 60-second, 5-minute, and 15-minute windows. History is stored in a bounded deque (100,000 events).

4.3 VWAP and Deviation Bands

Session VWAP computed via online running sums of price×volume and price²×volume, auto-resetting at session boundaries. Exposes real-time VWAP, deviation, and rolling standard deviation for band construction.

4.4 Volume Analysis

Tick-rule classifier for buy/sell trade direction inference. Volume profile engine buckets traded volume at each price level. Cumulative signed delta tracks net directional pressure.

4.5 Market Impact Estimation

Kyle's Lambda: OLS slope of price changes on signed order flow, quantifying permanent price impact per unit of informed trading. Amihud Illiquidity Ratio: average ratio of absolute returns to rupee volume.

5. Anomaly Detection

The anomaly detection module runs online rolling statistics (mean, standard deviation, z-score) over configurable sliding windows and flags the following events:

- **Spread blow-ups** — quoted spread exceeds 3σ above the 300-second rolling mean
- **Volume spikes** — tick volume exceeds 3× the 300-second rolling average
- **OFI extremes** — absolute order flow imbalance ratio exceeds 5.0 relative to near-touch quantity

Detected anomalies are timestamped, classified by severity, and broadcast to dashboard clients via WebSocket.

6. Data Pipeline

A pluggable data source abstraction supports switching between a mock generator and live Angel One SmartAPI feeds via a single environment variable (DATA_SOURCE=mock|angel). The mock source produces mean-reverting prices with geometric Brownian motion noise, imbalance-biased order books, and stochastic volume spikes.

Raw tick data is written to Apache Parquet files with a buffered writer that flushes every 500 rows or on partition rollover, using thread-safe locking. The API layer provides REST endpoints for historical data and WebSocket endpoints for real-time streaming, orchestrated by a singleton Streamer.

7. Frontend Dashboard

The React dashboard connects to three WebSocket channels (order book, analytics, alerts) using a custom auto-reconnecting hook with exponential backoff. The interface consists of six panels:

- Order Book — 5-depth bid/ask with proportional quantity bars
- Statistics Card — LTP, midprice, spread, OFI, VWAP, cumulative delta
- Price + VWAP Chart — candlestick overlay with deviation bands
- Spread Time Series — real-time spread evolution
- OFI Chart — rolling order flow imbalance with zero-line reference
- Anomaly Log — reverse-chronological alert feed with severity colouring

8. References

[1] Cont, R., Kukanov, A. & Stoikov, S. (2014). *The Price Impact of Order Book Events.* Journal of Financial Econometrics, 12(1), 47–88.

[2] Kyle, A.S. (1985). *Continuous Auctions and Insider Trading.* Econometrica, 53(6), 1315–1335.

[3] Amihud, Y. (2002). *Illiquidity and Stock Returns: Cross-Section and Time-Series Effects.* Journal of Financial Markets, 5(1), 31–56.

[4] Lee, C.M.C. & Ready, M.J. (1991). *Inferring Trade Direction from Intraday Data.* Journal of Finance, 46(2), 733–746.

9. Repository

GitHub: https://github.com/aariiparekh3012-collab/quantproject2

Aarya Parekh

B.Tech Student, Indian Institute of Technology Bombay

Date: 16 August 2026

Document Hash: SHA-256:D155BAC28411FCF70E1D140A

Timestamp: 2026-08-16T00:00:00+05:30

This document was prepared by the author and constitutes an accurate technical description of the project as of the date indicated above.